

React Könyvkezelő Alkalmazás

Web Technológiák 1.

Tóth Zsombor Gábor

D0H157

Tartalomjegyzék

1. Bevezetés és a projekt célja	2
1.1. Választott technológiák	2
2. Kezdeti lépések és a környezet kialakítása	3
2.1. Fájlstruktúra kialakítása	3
3. Az alkalmazás logikájának felépítése	4
3.1. Komponensek létrehozása	4
4. Fejlesztési kihívások és megoldások	5
4.1. A "Dupla Űrlap" probléma	5
4.2. Eseménykezelési hiba (Event Bubbling)	5
5. Végző simítások és Refaktorálás	6
5.1. Sötét mód (Dark Mode) véglegesítése	6
5.2. Értesítések és Reset gomb	6
6. Összegzés	7

1. Bevezetés és a projekt célja

A projekt célja egy modern, reszponzív webes alkalmazás létrehozása volt, amely lehetővé teszi a felhasználók számára olvasmányaik nyilvántartását. Az alapvető követelmény egy úgynevezett CRUD (Create, Read, Update, Delete) rendszer megvalósítása volt, ahol a felhasználó új könyveket adhat hozzá, listázhatja őket, szerkesztheti az adataikat, és törölheti azokat.

A fejlesztés során kiemelt szempont volt a felhasználói élmény (UX) és a tiszta kód-bázis, ezért a React keretrendszert választottam a modern komponensalapú fejlesztés érdekében.

1.1. Választott technológiák

- **React 18:** A felhasználói felület építéséhez.
- **Vite:** A gyors fejlesztői környezet és build folyamat biztosítására.
- **Tailwind CSS:** A gyors és konzisztens stílusozás érdekében.
- **LocalStorage:** Az adatok perzisztens tárolásához, backend nélkül.

2. Kezdeti lépések és a környezet kialakítása

A fejlesztést a projekt inicializálásával kezdtem. A modern szabványoknak megfelelően a Vite-ot használtam a create-react-app helyett, mivel sokkal gyorsabb indulást és frissítést tesz lehetővé.

```
1 npm create vite@latest konyv-app -- --template react
2 cd konyv-app
3 npm install
4 npm install -D tailwindcss postcss autoprefixer
5 npx tailwindcss init -p
6
```

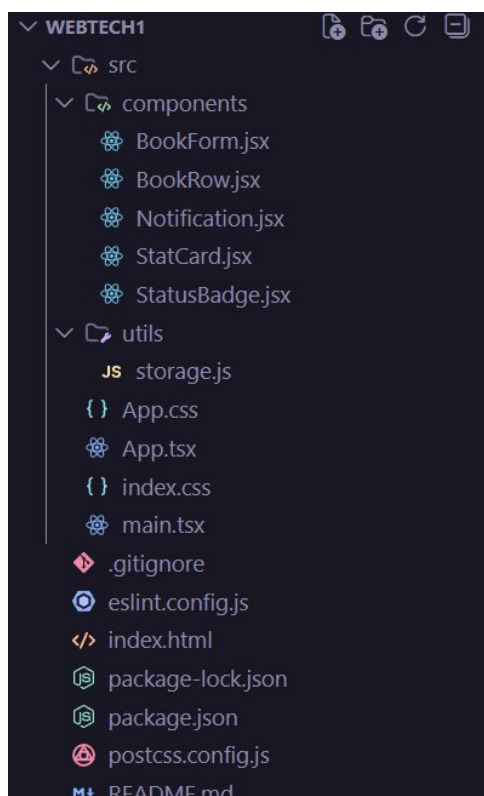
Listing 1. A projekt létrehozása terminálban

A telepítés után konfiguráltam a Tailwind CSS-t, hogy felismerje a React komponenseket, és beállítottam a globális stílusokat a sötét téma (Dark Mode) támogatására, mivel ez volt a preferált vizuális irány.

2.1. Fájlstruktúra kialakítása

Kezdetben egyetlen fájlban próbáltam megírni a logikát, de hamar rájöttem, hogy ez nem fenntartható. Ezért létrehoztam egy moduláris struktúrát:

- **src/components/**: Itt tárolom a vizuális elemeket (pl. BookRow, StatCard).
- **src/utils/**: Ide kerültek a segédfüggvények, például a localStorage kezelése.



1. ábra. A projekt végleges könyvtárszerkezete

3. Az alkalmazás logikájának felépítése

Az alkalmazás gerincét a `App.jsx` fájl adja, amely a fő állapotokat (state) kezeli. A legnagyobb kihívást az adatok szinkronban tartása jelentette a `localStorage`-dzel.

Ehhez létrehoztam egy `storage.js` segédfájlt, ami biztosítja, hogy az oldal újratöltésekor ne vesszenek el az adatok.

```
1 export function loadBooks() {
2   try {
3     const raw = localStorage.getItem(STORAGE_KEY);
4     if (!raw) return initialBooks;
5     const parsed = JSON.parse(raw);
6     return Array.isArray(parsed) ? parsed : initialBooks;
7   } catch (e) {
8     console.warn('Hiba az olvasaskor:', e);
9     return initialBooks;
10  }
11 }
12
```

Listing 2. Adatok betöltése LocalStorage-ból

3.1. Komponensek létrehozása

A felhasználói felületet kisebb, újrafelhasználható egységekre bontottam:

1. **BookRow:** Egyetlen könyv adatait jeleníti meg a listában. 2. **BookForm:** Ez felelős az új könyv felvételéért és a meglévők szerkesztéséért. 3. **StatCard:** Statisztikai adatokat mutat (pl. elolvasott könyvek száma).



2. ábra. A könyvlista megjelenítése komponensekkel

4. Fejlesztési kihívások és megoldások

A fejlesztés során több problémával is szembesültem, amelyeket iteratív módon javítottam.

4.1. A "Dupla Űrlap" probléma

Kezdetben a szerkesztés gombra kattintva egy új űrlap nyílt meg a lista alatt, ami vizuálisan zavaró volt és nehezen átláthatóvá tette az oldalt.

Megoldás: Átalakítottam az elrendezést egy "Sticky Sidebar" (ragadós oldalsáv) megoldásra. Így a képernyő jobb oldalán mindig látható maradt az űrlap. Ha a felhasználó rákattint egy könyvre a listában, az űrlap automatikusan kitöltődik az adott könyv adataival (Edit mód), egyébként pedig új könyv felvételére szolgál (Create mód).

```
1 // A form automatikusan tudja, hogy szerkesztünk vagy újat adunk
   hozzá
2 useEffect(() => {
3   if (initialData) {
4     setFormData({ ...initialData }); // Szerkesztes mod
5   } else {
6     setFormData({ title: '', ... }); // Uj hozzaadasa mod
7   }
8 }, [initialData]);
```

Listing 3. Az urlap logikajanak egyszerusitese

4.2. Eseménykezelési hiba (Event Bubbling)

Egy érdekes hiba volt, hogy a "Törlés" gombra kattintva az alkalmazás nemcsak törölte a könyvet, hanem egy pillanatra megpróbálta megnyitni szerkesztésre is. Ez azért történt, mert a kattintás eseménye "felbuborékolt" a szülő elemre (a sorra).

Megoldás: Bevezettem az `e.stopPropagation()` használatát a gomboknál.

```
1 const handleAction = (e, action) => {
2   e.stopPropagation(); // Megallitjuk az esemenyt
3   action(book);
4 };
5
```

Listing 4. Event propagation megallitasa

5. Végző simítások és Refaktorálás

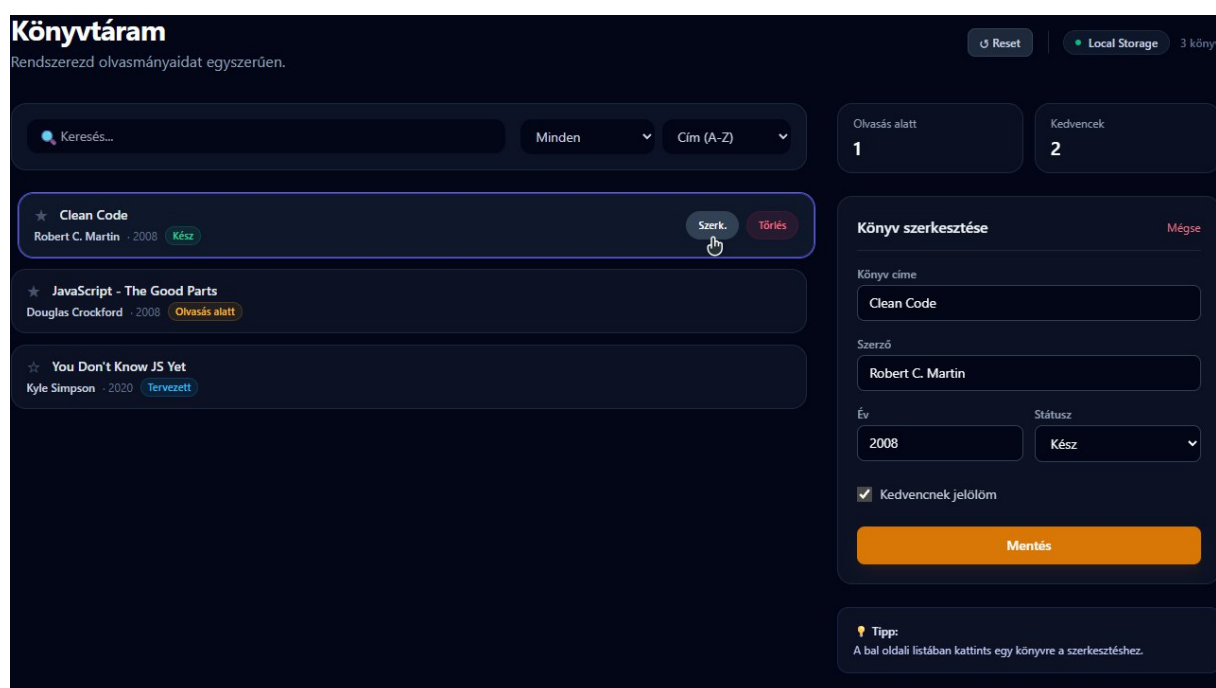
Miután a funkciók működtek, a kód tisztítására és a design véglegesítésére koncentráltam.

5.1. Sötét mód (Dark Mode) véglegesítése

A Tailwind segítségével egy modern, sötétszürke (`slate-950`) alapot adtam az alkalmazásnak, ami kíméli a szemet és professzionális hatást kelt.

5.2. Értesítések és Reset gomb

Hozzáadtam egy `Notification` komponenst, hogy a felhasználó visszajelzést kapjon (pl. "Sikeres mentés"). Továbbá bekerült egy "Reset" gomb is, amellyel visszaállítható az eredeti mintaadatbázis, ami hasznos a teszteléshez.



3. ábra. A kész alkalmazás felülete sötét módban

6. Összegzés

A projekt során sikerült egy teljes értékű React alkalmazást felépítenem a nulláról. Megtanultam a komponens-alapú gondolkodást, a `State` és `Effect` hookok helyes használatát, valamint a modern CSS keretrendszerek integrálását. Az alkalmazás stabil, gyors, és a refaktorálásnak köszönhetően a kódja jól karbantartható.